

FAQ

Практические вопросы про использование PhotoArchive — рассчитано на то, что вы не читали `README.pdf` и просто хотите понять, как этим пользоваться. Если ответа на свой вопрос здесь не найдёте — там же описано подробнее. Все примеры ниже реально прогонялись через программу при написании, а не придуманы.

Ниже периодически встречаются слова **SOURCE** и **TARGET** — так называются, соответственно, папка с исходными фото и папка с самим архивом (в основном тексте — «источник» и «архив»).

Основное

Что вообще делает эта программа?

Собирает ваши фото и видео (с диска, флешки, старого бэкапа, скачанного архива) в один упорядоченный архив в новом месте — раскладывает по альбомам (если папки были осмысленно названы вами) или по датам съёмки (если нет), убирает точные повторы, определяет дату по метаданным камеры. Всё это — **копированием**: программа читает исходные файлы и пишет их копии в указанное вами новое место (архив), не трогая и не перемещая исходную папку. Как именно программа решает, что из этого — альбом, а что разложить по дате, — ключевой навык для работы с программой, разобран отдельно, сразу в следующем разделе.

Она может что-то испортить или удалить в моих исходных фото?

Нет. Программа только читает файлы источника — переименовать, переместить или удалить что-либо там она не умеет технически: путь записи внутри исходной папки нигде в программе не формируется. Оригиналы остаются ровно в том виде, в каком были. Техническое обоснование — `README.pdf`, раздел [«SOURCE — только для чтения»](#).

Нужен ли для этого интернет?

Нет. Вся обработка — на этом же компьютере, ничего никуда не отправляется. Место (город), которое иногда добавляется к названию папки при раскладке по датам, определяется из внутренних данных самого файла.

Как запустить программу в первый раз?

Запустите `PhotoArchive.exe`. Появится меню из четырёх пунктов — [1] посмотреть, что нашлось (ничего не пишет), [2] посмотреть, что было бы сделано при реальной сборке (тоже ничего не пишет), [3] собрать по-настоящему, [4] заново проверить уже собранный архив (см. [«Что такое „Паспорт](#)

[архива»](#) ниже). Для первого раза разумно пройти по порядку [1] → [2] → [3], чтобы увидеть результат заранее, ничем не рискуя. Подробный пошаговый разбор — [QUICKSTART.pdf](#).

Где посмотреть список всех команд и параметров?

Запустите `PhotoArchive.exe --help` (или, например, `PhotoArchive.exe archive --help` — для параметров конкретного режима) прямо из командной строки — программа напечатает полный список подкоманд и флагов.

Как понять, что архив собрался успешно?

В конце работы в консоли (и в файле `summary.txt` внутри архива) печатается итог: сколько файлов обработано, сколько фото/видео, сколько дублей, сколько не удалось прочитать или уверенно датировать. Если что-то пошло не так — там же явное сообщение, что именно (например, кончилось место), а не тихий обрыв на середине.

Можно ли прервать работу на середине и запустить заново?

Да, в любой момент — Ctrl+C или просто закрыть окно. Архив устроен так, что недописанный файл в него попасть не может (сначала пишется под временным именем, переименовывается в финальное только после проверки целостности). Повторный запуск сам разберётся, что уже собрано, и продолжит, а не начнёт всё заново и не задвоит уже дописанное.

Как программа решает, класть фото в альбом или по дате?

Один из самых важных разделов этого FAQ. Неверное представление об этом правиле не портит и не удаляет исходные фото (SOURCE программа только читает — см. [«Она может что-то испортить или удалить в моих исходных фото?»](#) выше), но может дать архив с неправильной структурой на реальном объёме файлов (тысячи) — а вручную реорганизовать уже собранный архив потом дорого, особенно если исходная папка к тому моменту уже не хранится отдельно (см. [«Результат сборки получился не таким, как хотелось?»](#) ниже). Дешевле всего проверить ожидания заранее, через [1] / [2] меню (ничего не пишут на диск), чем разбираться с этим постфактум.

Здесь — только базовое правило и самые частые случаи. Полный список того, что настраивается через `photoarchive_config.yaml`, а что зашито в коде намертво, — в `README.pdf`, раздел [«Маршрутизация: альбом или по дате»](#).

Коротко, по шагам — дальше в разделе каждый пункт разобран отдельно, с примерами:

1. Программа идёт от корня источника вниз и проверяет КАЖДЫЙ сегмент пути к файлу, включая собственное имя архива, если файл внутри `.zip` / `.rar` / `.7z` / `.tar`.

2. Если ЛЮБОЙ сегмент — служебный (похож на автосозданную папку камеры/облака, голое число любой длины, папка с ~ в начале имени и т.п.) — файл целиком уходит по дате, в `ByDate\...`, БЕЗ исключений по тому, где на пути встретился этот сегмент (в самом начале, в середине, прямо перед файлом — не важно).
3. Если ни один сегмент не служебный — путь целиком становится альбомом в `Albums\...`, с сохранением всей структуры один в один. Каждая папка на этом пути — свой собственный, отдельный альбом.
4. Если один и тот же файл (байт-в-байт) лежит и в альбоме, и в служебной папке — в архив попадёт ровно одна копия, в альбом: осмысленные альбомы программа всегда обрабатывает раньше, чем всё остальное.

Важно: правило одинаковое на КАЖДОМ уровне вложенности, программа не «ищет самое подходящее имя» и не «заглядывает глубже» — один служебный сегмент решает исход для всего, что ниже него, безвозвратно. Если у вас `Отпуск 2015\Море\IMG.jpg` (ни один сегмент не служебный) — оба уровня сохранятся как есть: `Albums\Отпуск 2015\Море\IMG.jpg`, и оба — «Отпуск 2015», и «Море» — считаются отдельными альбомами.

Сквозной пример: несколько правил сразу

Каждое правило по отдельности разобрано ниже на своём примере — но на практике они действуют одновременно, на одном и том же дереве папок. Один и тот же условный источник:

```
SOURCE\  
Отпуск 2015\Море\IMG_0001.jpg  
Отпуск 2015\~Яндекс_диск\Резервная\IMG_0002.jpg  
Отпуск 2015\видео.zip (внутри архива: клип.mp4)  
DCIM\IMG_0001.jpg (та же фотография байт-в-байт, что и выше)
```

даёт четыре разных исхода четырьмя разными путями:

```
Отпуск 2015\Море\IMG_0001.jpg  
→ Albums\Отпуск 2015\Море\IMG_0001.jpg  
(ни один сегмент не служебный — путь целиком становится альбомом, «Отпуск 2015» и «Море»  
сохраняются как есть)  
  
Отпуск 2015\~Яндекс_диск\Резервная\IMG_0002.jpg  
→ ByDate\...\IMG_0002.jpg  
(папка с «~» на пути — отравление: всё, что лежит ниже, уходит по дате, даже «Резервная»  
с виду обычным именем)  
  
Отпуск 2015\видео.zip → клип.mp4  
→ Albums\Отпуск 2015\видео\клип.mp4  
(собственное имя архива не служебное — участвует в зеркалировании на равных с папкой,  
см. вопрос про архивы ниже)  
  
DCIM\IMG_0001.jpg (та же фотография, что и в первой строке)  
→ не копируется отдельно — уже есть в Albums\Отпуск 2015\Море\IMG_0001.jpg
```

(сам по себе DCIM служебный, файл ушёл бы по дате — но это точная копия того, что уже попало в альбом строкой выше: альбомы обрабатываются раньше, побеждает уже занятое место)

Какие папки считаются служебными, и можно ли это изменить?

Два рода служебных имён:

- **Настраиваемые** — автосозданные папки камер/облаков/скриншотов/мессенджеров (`DCIM` , `Camera` , `Camera Roll` , `Screenshots` , `WhatsApp*` , `Downloads` , `Desktop` , `Pictures` и т.п.). Список редактируется через `dump_segment_names` / `extra_dump_segment_names` в `photoarchive_config.yaml` .
- **Всегда служебные, менять нельзя** — защита архива от самопоедания: собственные корни архива (`ByDate` , `Albums` , `RAW` , `_Unsorted`), тер `[PhotoArchive]` и ещё несколько похожих технических случаев — полный список в `README.pdf`, раздел [«Маршрутизация: альбом или по дате»](#).

Если готового списка не хватает (например, своя папка синхронизации `MyCloudSync` , которую вы всегда хотите раскладывать по дате) — допишите её в `extra_dump_segment_names` , не переименовывайте руками каждый раз. Остальные настраиваемые параметры (гранулярность дат, порог маленьких изображений, режим определения места и т.д.) — `README.pdf`, раздел [«Файл конфигурации photoarchive_config.yaml »](#).

Как заставить программу принудительно НЕ считать конкретную папку альбомом?

Переименуйте её, добавив `~` в начало имени: `Яндекс_диск` → `~Яндекс_диск` . Программа никогда не переименовывает файлы источника сама — это вы один раз меняете имя своей папки, дальше её содержимое раскладывается по дате, как будто папки с таким именем вообще не было.

<code>Яндекс_диск\photo.jpg</code>	→ <code>Albums\Яндекс_диск\photo.jpg</code>	(выглядит как настоящий альбом)
<code>~Яндекс_диск\photo.jpg</code>	→ <code>ByDate\...\photo.jpg</code>	(тильда принудительно делает папку служебной)

Полезно для папок с правдоподобным именем (например, облачная синхронизация), которые вы не хотите видеть отдельным альбомом. Если это регулярно повторяется на разных запусках — рассмотрите `extra_dump_segment_names` в конфиге вместо ручного переименования каждый раз.

Правило одинаковое НЕЗАВИСИМО от того, где на пути встретилась служебная папка — например, синхронизированная папка облака оказалась вложена внутрь папки-альбома (`Отпуск 2015\~Яндекс_диск\photo.jpg`): тильда не просто «стирает» саму эту папку из результирующего пути — она безвозвратно отменяет альбом для ВСЕГО, что лежит глубже неё. Файл уйдёт по дате (`ByDate\...`), а не в `Albums\Отпуск 2015\photo.jpg` , даже если ещё глубже внутри этой же служебной папки встретится подпапка с вполне осмысленным именем — назад дороги нет, повторный поиск нового имени альбома после точки, где встретилась тильда/служебное имя, не выполняется. Это же правило действует и на самом верху пути: если источник указывает прямо на служебную папку

(например, целиком на `Pictures` или на профиль `Users\Имя`), весь путь падает по дате, даже если внутри лежит вполне осмысленно названный альбом — программа не «заглядывает глубже» в поисках настоящего имени, единственное решение — указать `--source` на уровень глубже, прямо на нужную папку.

Какие типичные папки НЕ станут именем альбома — и что будет вместо этого?

Голое число (в том числе дата вида `2015-08-20` — в ней тоже нет букв, и день-папка вида `20240802`) само по себе никогда не становится именем альбома, и, как и любое другое служебное имя, отправляет всё, что глубже него, без исключений по тому, где на пути оно встретилось:

Путь в источнике	Результат
<code>12345\photo.jpg</code> (голое число)	служебное → <code>ByDate\...</code>
Класс <code>12345\photo.jpg</code> (есть буква)	<code>Albums\Класс 12345\photo.jpg</code>
Альбом\DCIM\photo.jpg (камера-папка уже внутри альбома)	DCIM отменяет альбом для всего, что глубже → <code>ByDate\...\photo.jpg</code>
Альбом\20240802\photo.jpg (день-папка уже внутри альбома)	тоже отменяет альбом → <code>ByDate\...\photo.jpg</code>
<code>Users\Имя\СемейныйАрхив\photo.jpg</code> (весь профиль Windows как источник)	<code>Users</code> служебное → <code>ByDate\...\photo.jpg</code> , СемейныйАрхив не спасает — см. выше

Если у вас голое число — это осмысленное имя (номер комнаты, позывной), решение то же, что и с тильдой выше, только в обратную сторону: один раз допишите в имя папки хотя бы одну букву (`12345` → `Класс 12345`).

Источник — общая папка с несколькими альбомами внутри. Они не склеятся в один?

Нет. Каждая содержательная (не служебная, с буквами) подпапка внутри найденной ветки — это её ОТДЕЛЬНЫЙ вложенный альбом, независимо от того, на какую папку указывает `--source`:

```
SOURCE = D:\Мои фото, внутри: Мои фото\Свадьба\a.jpg и Мои фото\Отпуск\b.jpg
Результат: Albums\Мои фото\Свадьба\a.jpg и Albums\Мои фото\Отпуск\b.jpg (ДВА альбома – «Свадьба» и «Отпуск», не один общий «Мои фото»)
```

Правило одинаковое на любом уровне вложенности — оно в равной мере разделяет и подпапки одного события (`Свадьба 2015\Церемония` и `Свадьба 2015\Банкет` — тоже два разных альбома, не подпапки одного «Свадьба 2015»). Указывать `--source` отдельно на каждый альбом не нужно — общая папка-родитель сама не становится альбомом, просто общим предком в дереве `Albums\`.

Что значит файл `__ПРОПУЩЕННЫЕ_ДУБЛИ.txt` внутри альбома?

Следствие того же деду па по содержимому, что и в вопросе «Куда деваются повторяющиеся фото?» (см. «Типичные сценарии» ниже), только на уровне одного альбома: если один и тот же кадр случайно скопирован в исходную папку альбома дважды под разными именами, физически в архив попадёт только одна копия. Маркер-файл называет пропущенный дубль:

```
Albums\Свадьба 2015\  
__ПРОПУЩЕННЫЕ_ДУБЛИ.txt  
IMG_0001.jpg
```

```
IMG_0001_copу.jpg -- совпадает по содержимому с IMG_0001.jpg
```

Файл появляется только когда в альбоме реально есть хотя бы один такой пропуск, свободно удаляется (программа никогда его не читает) и перезаписывается целиком на каждом прогоне — не хранит историю между прогонами. Два РАЗНЫХ альбома (например, общие кадры со стороны жениха и невесты на разных по имени/пути свадебных папках) физически никогда не делят одну и ту же папку архива — этот маркер-файл не про совпадения МЕЖДУ разными альбомами, только про дубли внутри одного и того же. Подробнее — README.pdf, раздел [«Пропущенные дубли внутри альбома»](#).

Один и тот же файл лежит и в настоящем альбоме, и в «мусорной» папке — что попадёт в архив?

Побеждает альбом — всегда, независимо от того, какая из двух папок должна была обойтись раньше по алфавиту. Если в источнике одна и та же фотография есть, например, и в `Отпуск 2015\photo.jpg` (настоящий альбом), и в `DCIM\photo.jpg` (служебная папка камеры, разложилась бы по дате) — в архив попадёт ровно одна копия, в `Albums\Отпуск 2015\photo.jpg`. Копия из `DCIM` будет распознана как уже присутствующая в архиве (`skipped.csv` , `already_present`), вторая физическая копия не создаётся. Раньше исход такой гонки зависел от порядка обхода (какая папка стоит раньше по алфавиту) — теперь программа всегда сначала полностью обрабатывает осмысленные альбомы и только потом — служебные/ `~` -папки, поэтому осознанно организованный альбом больше не может случайно проиграть автоматической раскладке по дате.

Ограничение: эта гарантия действует только в пределах одного прогона. Архив пополняется по принципу «только дописывать» — если та же фотография уже физически осела в `ByDate\...` в одном из прошлых пополнений, а сейчас та же фотография встретилась в настоящем альбоме, старая копия по дате не переносится в альбом задним числом.

Как обрабатываются альбомы из `zip/rar/7z/tar` архивов?

Архивный файл — эквивалент папки: собственное имя архива (без расширения) проверяется тем же правилом, что и имя любой папки, на своей позиции в пути, наравне со всем остальным:

Свадьба\видео.zip → фото.jpg (внутри)	→ Albums\Свадьба\видео\фото.jpg
Загрузки\export.zip → archive\фото.jpg	→ Albums\export\archive\фото.jpg

В первом примере имя архива (видео) — не служебное, становится просто ещё одним уровнем пути внутри уже найденного альбома Свадьба — **всегда**, даже если внутри архива лежит единственный файл (без исключения по количеству). Во втором примере Загрузки — служебное имя, но само оно не входит в путь (первый не-служебный сегмент — имя архива export), поэтому альбомом становится сам архив, внутренняя структура (включая папку archive внутри него — тоже не служебное имя) сохраняется как есть.

Если собственное имя самого архива — служебное или начинается с ~ (например, Свадьба\~backup.zip) — архив ведёт себя как независимый объект: всё его содержимое уходит по дате (ByDate\...), даже если сам архивный файл лежит прямо внутри настоящего альбома. Это не портит соседние файлы рядом с архивом — они размещаются как обычно, будто архива не было вовсе (у них в пути просто нет этого сегмента). Тот же приём, что и с обычной папкой (см. вопрос про ~ выше), просто применённый к самому архивному файлу, а не к папке.

Типичные сценарии

У меня старый диск/флешка с фото за много лет вперемешку — с чего начать?

Укажите этот диск/папку как источник и запустите сначала пункт [1] (сканирование) — покажет, сколько всего фото/видео нашлось и сколько места они занимают, ничего не копируя. Дальше [2] (пробный прогон) покажет, что именно получилось бы: сколько в альбомы, сколько по датам, сколько дублей. Только после этого — [3] , реальная сборка.

Если число альбомов заметно меньше ожидаемого (в пределе — ноль, всё ушло по датам) — это сигнал, что какой-то сегмент пути (например, имя самой папки-источника или что-то у неё внутри) распознан программой как служебный и отправил всё, что ниже — см. «Маршрутизация: альбом или по дате» в README.pdf. Проверьте имена папок до перехода к [3] , а не после.

Позже, когда архив уже собран и вы просто хотите время от времени убедиться, что с ним всё в порядке (без нового источника под рукой) — для этого отдельный пункт [4] , см. следующий раздел.

У меня фото в нескольких папках/на нескольких дисках — это один запуск или несколько?

Можно и так, и так. --source можно указать несколько раз за один запуск (--source "D:\Фото2015" --source "E:\Архив старый") — все обработаются в один и тот же архив за один прогон. Либо запускать по очереди на один и тот же архив — дубли между запусками не появятся, программа каждый раз сверяется с тем, что уже реально лежит в архиве. Ещё есть --source-list (список путей

файлом) и `--source all` (сразу все локальные диски) — README.pdf, раздел [«Параметры командной строки»](#).

Мой источник или архив лежит на сетевом диске — это поддерживается?

Да, но не через быстрое подменю. В подменю выбора пути (пункты [1] - [4]) показываются только локальные диски — фиксированные, съёмные и оптические; замапленные сетевые диски там не перечисляются. Выберите в том же меню пункт «своя папка» и введите путь вручную (буквой замапленного диска или сетевым путём вида `\\сервер\папка`) — дальше программа работает с ним точно так же, как с локальным. Учитывайте только, что скорость операций (особенно проверки целостности) на сетевом диске упирается в скорость сети, а не диска, и может быть заметно ниже, чем на локальном накопителе.

Для обычного анализа ([1] /CLI `analyze`) на медленном/сетевом диске можно дополнительно ускорить проверку, отключив сверку сигнатуры файла (`check_signature: false` — так и есть по умолчанию, менять ничего не нужно) в `photoarchive_config.yaml` — эта проверка читает начало каждого файла отдельно и заметнее всего именно на сетевых дисках. «Паспорт архива» ([4]) эту проверку не сокращает никогда, независимо от настройки — там она нужна для полноты.

Куда деваются повторяющиеся фото?

Точная копия (тот же файл, пришедший, например, с двух разных дисков) не копируется повторно — она там уже есть, тратить место незачем. Сравнение идёт только по содержимому файла, имя роли не играет: если два файла с разными именами байт-в-байт одинаковы, в архив попадёт только тот, что обнаружился первым, остальные — точные дубликаты, что бы у них ни было написано в названии. Это происходит автоматически, ничего настраивать не нужно.

А если несколько почти одинаковых кадров подряд — например, серийная съёмка?

Такие кадры («near-dup») — не то же самое, что точный дубликат: программа не уверена, отличаются ли они существенно, поэтому **сохраняет все**, не выбирая «лучший». Серия из 40 кадров подряд (частая при серийной съёмке на телефоне) останется всеми 40 — лишнюю копию всегда можно стереть вручную, а решающий кадр так не потерять. Подробнее — README.pdf, раздел [«Дедупликация»](#).

У меня есть видео с DVD (папка VIDEO_TS) — программа их тоже заархивирует?

Да. Папка с типичной структурой DVD-Video (VIDEO_TS с файлами `.vob` / `.ifo` / `.bup`) распознаётся отдельно и копируется в архив целиком, как единое целое — не по отдельным файлам внутри неё. Повторное сканирование того же диска не создаёт вторую копию: программа запоминает, какие диски уже заархивированы, и при следующем прогоне просто пропускает их. Подробнее — README.pdf, раздел [«DVD-видео \(VIDEO_TS\)»](#).

Кончилось место на диске прямо во время сборки — что теперь?

Программа остановится с понятным сообщением, ничего не повредив (архив к этому моменту цел — см. выше про временные файлы). Освободите место на диске, где собирается архив, и запустите ту же команду ещё раз — прогон продолжит с того, что уже дописано, не с начала. Подробнее, в т.ч. как заранее увеличить запас свободного места (`free_space_margin_gb`) — README.pdf, раздел [«Диагностика и устранение неполадок»](#).

Программа как будто зависла — счётчик/бар подолгу не двигается. Это баг?

Смотря на каком шаге. Программа всегда идёт по одному и тому же порядку шагов — см. README.pdf, раздел [«Как работает программа: фазы одного прогона»](#) — и в двух местах этого порядка долгая пауза ожидаема, не сбой:

- **«Фаза 1 — индексация архива»** (перед началом обработки источника, только у режимов, сверяющихся с TARGET). Программа читает содержимое уже собранного архива целиком, чтобы знать, с чем сравнивать дубликаты — на большом архиве (десятки/сотни тысяч файлов) это может занимать заметное время, особенно на первом прогоне. Здесь есть и %, и оценка оставшегося времени (ETA) — если они видны и медленно, но верно растут/уменьшаются, это ожидаемая работа, не сбой. Повторные прогоны того же архива обычно заметно быстрее (кэш).
- **«Фаза 2-5 — обработка источника»**. Строка статуса здесь показывает «объектов X/Y» — честный счётчик обработанного по сравнению с быстрой предварительной оценкой всего источника, не точный процент и не оценка оставшегося времени (сам источник ещё обходится «по ходу», пока идёт эта фаза, — точное число заранее не известно, только прикидка). Если X понемногу растёт — всё идёт штатно, даже если это происходит медленно. Отдельно — когда рядом с счётчиком появляется текст «хеширование видеофайла (X ГБ)»: прежде чем поместить видео в архив, программа обязана прочитать его целиком с диска и посчитать контрольную сумму (SHA-256) — на объёмных видеофайлах (домашняя видеосъёмка, фильмы) это может занимать минуты на файл, а суммарно на архиве с большим числом таких файлов — часы.

В обоих случаях скорость зависит от скорости диска (особенно на внешних/сетевых дисках), не от программы — это цена самой проверки, не сбой. Подробнее — README.pdf, раздел [«Ограничения»](#).

Можно ли дописать новые фото в уже собранный архив позже?

Да, это штатный сценарий, не требующий отдельного «режима обновления». Запустите программу ещё раз с тем же источником (или новыми файлами в нём) и тем же архивом — уже собранное не задвоится, допишется только новое.

Ничто не мешает и вручную скопировать папку прямо в `Albums\ / ByDate\`, минуя программу — но в этом случае проверку на дубликаты она не увидит: дедуп-защита работает только для того, что проходит через сам прогон программы (см. README.pdf, раздел [«Как работает программа: фазы одного прогона»](#)). Если важно не получить случайный дубль — надёжнее добавлять новые файлы через саму программу, а не руками.

Рекомендуемый способ. Заведите отдельную папку-накопитель для новых фото между прогонами — **строго рядом** с архивом, а не внутри него: программа явно откажется запускаться, если источник указывает внутрь папки архива (иначе прогон мог бы повторно поглощать файлы, только что

записанные им же самим). Например, `D:\Архив` (архив) и `D:\Новое` (накопитель) — на одном уровне, рядом.

Имя накопителя может быть обычным, без `~` — каждая папка внутри него уже сама по себе свой отдельный альбом (см. «Маршрутизация: альбом или по дате» в README.pdf), несколько разных будущих альбомов внутри накопителя НЕ схлопнутся в один, никаких специальных мер для этого не требуется. `~` перед именем самого накопителя, наоборот, сейчас будет ЛИШНИМ и вредным — тильда отравляет всё, что внутри, целиком в ByDate (см. предыдущий вопрос про тильду) — то есть именно то, чего мы хотим избежать.

Дальше — раз в год или по мере накопления, обычный прогон с накопителем как источником:

```
PhotoArchive.exe --source "D:\Новое" --target "D:\Архив"
```

Перед первым таким прогоном на новом для вас накопителе не помешает `analyze` — он покажет, сколько альбомов программа реально увидела внутри. Если ожидали несколько, а видите один — самое время поправить имена папок до записи на диск, а не после.

Если папка уже оказалась внутри архива вручную (риск из предупреждения выше уже реализовался) — программа сама её не переклассифицирует и не допишет логи задним числом ни на каком следующем прогоне: Фаза 1 только читает содержимое архива, чтобы сравнивать с ним новые файлы, а не переносит и не пересортировывает то, что там уже лежит. Чтобы исправить — перенесите (не скопируйте) такую папку из архива в накопитель рядом с TARGET и прогоните обычным способом, как выше. Устаревшего кэша можно не бояться: Фаза 1 при каждом прогоне заново сканирует `Albums\` / `ByDate\` / `RAW\` с диска, а перенесённого файла там уже не будет — он пройдёт полноценный дедуп/классификацию/логи, как любой другой файл источника.

Что такое «Паспорт архива» и чем он отличается от обычного отчёта?

Это пункт `[4]` меню — отдельный режим, только для проверки уже существующего архива, без нового источника. В отличие от обычного отчёта после `[1]` / `[2]` / `[3]` (который опирается на то, что программа увидела при ЭТОМ конкретном прогоне SOURCE), `[4]` перечитывает сам архив (`Albums\` , `ByDate\` , `RAW\`) с нуля и заново ищет внутри него дубли, повреждённые файлы и файлы без определённой даты — независимо от истории прошлых запусков.

Спрашивает только TARGET (SOURCE ему не нужен вообще). Полезно в двух случаях: как обычная периодическая самопроверка «всё ли в порядке с архивом», и особенно — если в архиве что-то переносили, переименовывали или удаляли руками в обход программы (см. предупреждение в предыдущем вопросе про накопитель) — такие ручные правки обычный прогон не отслеживает, а `[4]` увидит их состояние как есть, заново пересчитав всё содержимое архива. У `[4]` пока нет отдельного CLI-режима — только пункт голого меню.

Результат сборки получился не таким, как хотелось — что делать?

Если это первая сборка (архив пока пополнялся только этим одним прогоном, а исходная папка/накопитель ещё цела) — можно спокойно удалить собранный архив (папку TARGET) целиком и запустить сборку заново с другими настройками (`photoarchive_config.yaml` — раскладка, гранулярность дат, список служебных папок и т.д.): оригиналы это не затронет (см. выше). Повторяйте сколько угодно раз, пока результат не устроит.

Если архив уже пополнялся НЕСКОЛЬКИМИ прогонами (см. [«Можно ли дописать новые фото в уже собранный архив позже?»](#) выше) — удалять его целиком, скорее всего, **нельзя без потери данных**. Программа не хранит копию исходных файлов отдельно от TARGET: если накопитель одного из прошлых прогонов уже очищен, удалён или переиспользован под другие файлы (обычная практика с такой папкой — не хранить её вечно), то единственная оставшаяся копия части фото — уже сам архив. Удаление TARGET в этом случае безвозвратно уничтожит и её. Не удаляйте архив, пока не убедитесь, что для КАЖДОГО прошлого пополнения либо сохранился исходный накопитель, либо результат вас и так устраивает. Если нужно только поправить раскладку части файлов — рассмотрите точечную правку руками внутри `Albums\` / `ByDate\`, а не пересборку с нуля.

Можно ли собрать архив из уже готового, ранее собранного архива?

Да — например, чтобы объединить несколько отдельных архивов в один, или пересобрать после смены настроек раскладки:

```
PhotoArchive.exe --source "D:\Старый архив" --target "D:\Новый архив"
```

Проверено на реальном прогоне: альбомы остаются альбомами, даты — датами, ничего не расползётся — служебные имена архива (`Albums` / `ByDate` / `RAW` / `_Unsorted` / `[PhotoArchive]` / `__служебные_файлы\`) защищены от того, чтобы самим стать именем нового альбома.

Где потом искать свои фото в готовом архиве?

Внутри указанного вами архива появятся две основные папки: `Albums\` — ваши осмысленно названные папки-альбомы (структура сохранена, как было у вас), и `ByDate\ГГГГ\ГГГГ-ММ\` — всё остальное, разложенное по дате съёмки. Как именно программа отличает одно от другого — см. раздел выше, [«Как программа решает, класть фото в альбом или по дате?»](#).

Смогу ли я смотреть и искать фото прямо в программе?

Нет, PhotoArchive не хранит и не показывает фото сама — результат работы это обычные папки с понятными именами, открываются любым приложением: стандартным просмотрщиком Windows, Проводником, любой программой для фото. Если нужен визуальный каталог с поиском — архив можно использовать как чистый, без дублей источник для него (см. [«Для чего ещё, кроме личного архива...»](#) про импорт в каталогизаторы вроде digiKam/Lightroom).

Куда делся файл? Логи и диагностика

Я точно знаю, что файл X был в источнике, но не нахожу его в архиве — где искать?

Смотрите по таблице — в зависимости от того, что произошло, файл (или строка про него) окажется в одном конкретном месте, не разбросан по нескольким. Все `.csv` — в `{TARGET}\__служебные_файлы\logs\` :

Исход	Где искать	Физическая копия?
Точный дубль уже есть в архиве	<code>skipped.csv</code>	Нет
Не уверены, реальное фото или нет (папка не «шумная»)	<code>_Unsorted\</code> + <code>disputes.csv</code>	Да
То же, но путь содержит <code>temp / cache / tmp / .thumbnails</code>	<code>rejected_noise.csv</code>	Нет
Ошибка чтения (файл был занят другим процессом и т.п.)	<code>unreadable.csv</code>	Нет
Служебная/исключённая папка при обходе (<code>node_modules</code> и т.п.)	только аргерат в <code>actions.log / summary.txt</code>	Нет
Нераспознанное расширение (<code>.txt</code> , <code>.docx</code> , <code>.exe</code> и т.п.)	нигде — не медиафайл	Нет

Пример реальных строк (пути сокращены):

```
skipped.csv:   Альбом/photo1_copy.jpg,<архив>\Альбом\photo1.jpg,already_present
disputes.csv:  Альбом/broken.jpg,unreadable_image,<архив>\_Unsorted\Альбом\broken.jpg,0
```

Что за файлы в `_Unsorted\` , и что с ними делать?

Это настоящие фотографии (не служебные данные), которые программа не смогла уверенно классифицировать — например, файл с валидным расширением, но повреждённым содержимым, вне «шумной» зоны. `_Unsorted\` — равноправная соседняя папка `Albums\ / ByDate\ / RAW\` , не защищена и не помечена как одноразовая — открывайте и разбирайтесь руками. Если позже (после обновления программы) файл удастся классифицировать увереннее — при повторном прогоне на этот же архив как источник он «дозреет» до нормального места автоматически, не задвоившись.

Что означают остальные `.csv` в `__служебные_файлы\logs\` ?

`skipped.csv / disputes.csv / rejected_noise.csv / unreadable.csv` — см. таблицу выше. Ещё два файла того же назначения (открывать нужно ещё реже — обычно только при явном разборе конкретного случая):

Файл	Что в нём
<code>appended.csv</code>	Каждый реально скопированный файл: откуда, куда, как именно.
<code>dates_review.csv</code>	Файлы, дата которых определена не по надёжному EXIF, — см. вопрос ниже.
<code>albums_merged.csv</code>	Честный повторный прогон дописывает уже существующий альбом (тот же путь источника) — см. также <code>__ПРОПУЩЕННЫЕ_ДУБЛИ.txt</code> выше.

У фото явно неправильная дата — как понять, откуда она взялась?

Смотрите `dates_review.csv` — туда попадает **каждый** файл, дата которого определена не по надёжному EXIF-метаданному камеры (Tier A), а по менее надёжному источнику: имени файла (Tier B) или по оценке (Tier C — медиана дат соседних файлов в той же папке). Реальная строка:

```
Альбом/DateCluster/nodate.jpg,2019-07-16T11:00:00,C,low,inferred_from_folder_cluster,...
```

Это файл без собственной даты, у которого соседи по папке датированы 2019-07-15 и 2019-07-17 (по надёжному EXIF) — программа взяла медиану. Колонки: путь, определённая дата, tier (A/B/C), уровень доверия, откуда взялась (`evidence`), исходные данные (`source`). Файлы вообще без какого-либо сигнала о дате (Tier D) в этот csv не попадают — они лежат в `ByDate\0000-undated\`, см.

[соответствующий раздел README.pdf](#).

Программа написала «N файлов не прочитано» — что это и что делать?

Это `unreadable.csv` — файлы, чтение которых не удалось даже после нескольких попыток с паузой (типичная причина — файл был в этот момент занят другим процессом, реже — реальная ошибка диска). Что делать: ничего специального — закройте программу, которая могла держать файл открытым, и запустите тот же прогон ещё раз; уже скопированное не задвоится, программа попытается дочитать именно то, что не получилось в прошлый раз.

Программа упала с ошибкой/закрылась — что делать?

Ищите `crash.log` в той же папке, где лежит сам `PhotoArchive.exe` (НЕ внутри архива) — он появляется только при неожиданной ошибке и содержит полный traceback. Приложите его при сообщении о проблеме (см. [«Как сообщить об ошибке»](#) ниже) — архив к этому моменту в безопасности в любом случае (см. выше про атомарную запись). Traceback почти всегда содержит полные пути (включая имя вашей учётной записи Windows и названия личных папок) — просмотрите файл перед тем, как прикладывать его к публичному Issue.

Как посмотреть точную причину конкретного решения (почему альбом/дамп, почему near-dup)?

Включите `debug: true` в `photoarchive_config.yaml` — в `actions.log` начнут писаться подробные `[DEBUG]`-строки с причиной каждого решения и полный traceback на ошибках (как и в `crash.log`, эти

строки могут содержать полные пути). Имейте в виду: разово включённый `debug` оставит эти строки вперемешку с обычными во всех последующих прогонах того же архива, пока вы не выключите флаг обратно.

Для чего ещё, кроме личного архива, может пригодиться эта программа?

Программа задумана как инструмент для личного архива (см. [«от автора»](#)), но по сути умеет безопасную offline-детерминированную гигиену данных — точный и перцептивный дедуп (включая near-dup/ кроп), датировку с явным провенансом, защищённую распаковку архивов и атомарную запись с hash-verify. Это может пригодиться и как отдельный этап **перед** чем-то более дорогим: перед импортом в каталогизаторы (digiKam, Lightroom и т.п.), перед сборкой датасетов для ML/CV (дедуп предотвращает утечку похожих кадров между train/val) или перед холодным архивным хранением.

Куда смотреть дальше

- [QUICKSTART.pdf](#) — пошаговый первый запуск.
- `README.pdf` — полный технический справочник по всем возможностям и настройкам, включая раздел «Маршрутизация: альбом или по дате» (формальное описание того же правила, которое здесь показано на практических примерах).
- `photoarchive_config.yaml.example` — все настраиваемые ключи с комментариями, а также справочный список названий папок, которые программа считает служебными без права редактирования.
- vo1012.github.io/PhotoArchive — сайт проекта.

Как сообщить об ошибке или предложить улучшение?

- **Предложения по развитию.** Если не хватает какой-то возможности, или что-то в поведении кажется неудобным — пишите через GitHub (github.com/vo1012/PhotoArchive, раздел Issues) или через раздел «Контакты» на сайте проекта.
- **Сообщения об ошибках.** Если что-то пошло не так — туда же, GitHub или сайт. Полезно приложить `crash.log` (лежит в той же папке, что и `PhotoArchive.exe`, появляется только при неожиданной ошибке — traceback внутри почти всегда содержит полные пути, включая имя вашей учётной записи Windows и названия личных папок) и/или файлы из `{TARGET}\__служебные_файлы\logs\` (обычные текстовые логи прогона — секретов сверх того, что уже было в исходных именах файлов и путей, они не содержат). В обоих случаях по желанию можно просмотреть и отредактировать перед отправкой.

Можно ли поддержать проект финансово? Это платная программа?

Нет, PhotoArchive бесплатна и останется такой для всех — использование не предполагает ни разовую оплату, ни подписку, никакие функции не скрыты за платой. Автор не занимается коммерческим распространением программы.

Если она окажется полезной и есть желание поддержать разработку — актуальный способ сделать это указан в разделе «Контакты» на сайте проекта.

Такой перевод не будет связан ни с какими дополнительными функциями и ничего не будет открывать — это будет именно добровольная благодарность, а не оплата.